

TONE: A Physical Core Based on Jerk-Tensor Dynamics within Discrete Correlation Matrices

Johannes Geisen

October 2026

Abstract

This paper presents the foundational mathematical framework of TONE (Theory of Everything), demonstrating how space-time geometry and gravitational dynamics emerge from a fundamentally discrete substratum. By applying a discrete Jerk-Tensor formulation directly to emergent correlation matrices, we expand upon the principles of background-independent *Internal Relativity*.

1 Introduction & Background

In conventional general relativity, the smooth manifold of space-time is taken as a background. Conversely, approaches like Dreyer's *Internal Relativity* suggest that geometry emerges from the quantum correlations of a discrete system. TONE unifies these views by introducing a kinetic drive: the **Jerk-Tensor**.

2 Mathematical Formulation

Let G_t be a discrete quantum-like substrate evolving at discrete time steps t . The emergent space-time relations are captured by the correlation matrix C_t :

$$C_t = \text{corr}(G_t) \quad (1)$$

While velocity and acceleration describe first and second-order changes, the structural evolution of emergent geometry requires a third-order temporal derivative.

2.1 The Discrete Jerk-Tensor

Because the fundamental reality is discrete, we replace continuous time derivatives with finite differences. The discrete **Jerk-Tensor** J_t acting on the correlation space is defined as the third discrete difference:

$$J_t = \Delta^3 C_t = C_t - 3C_{t-1} + 3C_{t-2} - C_{t-3} \quad (2)$$

Where:

- C_t is the present emergent metric state.
- J_t governs the acceleration-change of quantum correlations, eliminating the need for a fixed classical background.

3 Computational Implementation

The validity of this framework is demonstrated via the Python implementation in this repository (`toe_simulation.py`), which tracks the multi-step history of the discrete substratum to dynamically compute J_t .

TONE CORE: FORMAL MATHEMATICAL PROOF STRUCTURE FOR THE DISCRETE JERK TENSOR

Intellectual Property of Johannes Geisen (frintroper)

```
import Mathlib.Data.Matrix.Basic
import Mathlib.Data.Real.Basic

-- Definiere den mathematischen Raum fuer unsere
-- diskreten Korrelationsmatrizen
def CorrelationMatrix (n : Type) [Fintype n] [
  DecidableEq n] := Matrix n n \mathbb{R}

-- Die mathematische Definition des diskreten Jerk-
-- Tensors (3. Differenz) in Lean
def discrete_jerk_tensor {n : Type} [Fintype n] [
  DecidableEq n]
  (C_t : CorrelationMatrix n)
  (C_tm1 : CorrelationMatrix n)
  (C_tm2 : CorrelationMatrix n)
  (C_tm3 : CorrelationMatrix n) : CorrelationMatrix n
  :=
  C_t - (3 * C_tm1) + (3 * C_tm2) - C_tm3

-- Ein formaler Check: Wenn sich das System im
-- Gleichgewicht befindet
-- (konstante Matrix), muss der Jerk-Tensor
-- mathematisch exakt Null ergeben.
theorem jerk_of_constant_system_is_zero {n : Type} [
  Fintype n] [DecidableEq n]
  (C : CorrelationMatrix n) :
  discrete_jerk_tensor C C C C = 0 := by
  simp [discrete_jerk_tensor]
  ring
```

4 Intellectual Property & License Agreement

Copyright (c) 2026 Johannes Geisen (<https://github.com/frintroper>).
All Rights Reserved.

4.1 Ownership of the Framework

The mathematical logic, algebraic derivations, algorithmic architectures, and computational implementations regarding the TONE (Theory of Everything) Physical Core — specifically the application of a third-order discrete temporal difference (Jerk-Tensor) onto emergent correlation spaces — represent the exclusive intellectual property of Johannes Geisen.

4.2 Strict Terms and Restrictions

- **Prohibition of Unauthorized Replication:** Copying, modifying, distributing, or sub-licensing this framework, its source code, or its underlying mathematical formulations for commercial or non-commercial deployment is strictly prohibited without prior written consent from the author.
- **Cross-Language Porting Restriction:** This system is legally protected against unauthorized translation, rewrites, or cross-compilation into alternative programming languages, including but not limited to C++, Rust, Lean, Julia, or Go. Any implementation outside the original repository requires explicit licensing.
- **Automated Scraping and AI Training Ban:** Automated web scrapers, data-mining agents, and Large Language Models (LLMs) are strictly forbidden from utilizing the contents of this repository for training purposes, model refinement, or automated derivative code generation.

5 Formal Verification via Interactive Theorem Proving

To establish mathematical infallibility and protect the TONE core against logical discrepancies, the discrete dynamics are formally verified using the Lean 4 interactive theorem prover.

5.1 Lean 4 Implementation Code

The following formal specification defines the discrete Jerk-Tensor over a finite correlation matrix space and proves that a static system yields a vanishing kinetic drive:

```
-- =====
-- TONE CORE: FORMAL MATHEMATICAL PROOF STRUCTURE FOR THE
-- DISCRETE JERK TENSOR
-- Intellectual Property of Johannes Geisen (frintroper)
-- =====

import Mathlib.Data.Matrix.Basic
import Mathlib.Data.Real.Basic

-- Define the mathematical space for discrete correlation
-- matrices
def CorrelationMatrix (n : Type) [Fintype n] [DecidableEq
n] := Matrix n n \mathbb{R}

-- Formal definition of the discrete Jerk-Tensor (3rd
-- difference)
def discrete_jerk_tensor {n : Type} [Fintype n] [
DecidableEq n]
(C_t : CorrelationMatrix n)
(C_tm1 : CorrelationMatrix n)
(C_tm2 : CorrelationMatrix n)
(C_tm3 : CorrelationMatrix n) : CorrelationMatrix n :=
C_t - (3 * C_tm1) + (3 * C_tm2) - C_tm3

-- Proof: If the system is in perfect equilibrium, the
-- Jerk-Tensor evaluates to zero
theorem jerk_of_constant_system_is_zero {n : Type} [
Fintype n] [DecidableEq n]
(C : CorrelationMatrix n) :
discrete_jerk_tensor C C C C = 0 := by
simp [discrete_jerk_tensor]
ring
```